Trust systems: definitions and the first two lemmas

Date: 2026-07-04. Status: definitions + Lemmas 1–2 worked out with proofs and **Lean-verified** (`lean/Containers/Containers/Trust.lean`, builds with no sorry: `sound_le_tau`, `tau_sound`, `derivGrade_le_tau`, `tau_le_derivGrade`, `tau_transport`, and the circular counterexample `cyc_sound_top`/`cyc_no_deriv`/`cyc_not_wf`). **Theorem 3 (consolidation preservation) is now also Lean-verified** (`tau_consolidate`, plus the shape-only degradation counterexample `ground_tau_hi`/`layer4_tau_bot`); see §4 for the definition that avoids trivialization and §6 for the honest outcome of the preserved disagreement. **Consolidations compose** (`Consolidation.id`/`Consolidation.comp`/`tau_consolidate_comp`, Lean-verified): pipelines preserve τ layer-by-layer iff end-to-end (Theorem 4.3). The composition decided §6's remaining candidate — it is cofunctor composition, nothing comonadic — and a hand-trace of G13 through the loop experiment corrected §4's empirical anchor: the four-layer "degradation" is a *certificate-format* gap (certified t < canonical τ, Lemma 2.2 (ii)), not a loss of τ; the two failure modes converge one layer later. The formalization corrected the prose in three places, marked inline below: (1) T must be a chain for attainment; (2) morphisms must be strict at ⊥; (3) two case splits in the proof of Lemma 3.2. Companion artifacts: the three running deployments (Clio, MacBeth, the Warnaar loop experiment), the field report (`report-ghani-trust-citations.md`), and the directed-container formalization (`ProofSearchN.lean`, no sorry).

Everything here is in unicode; a .tex version comes later if the results hold.

---

1. The data of a trust system

Definition 1.1 (trust system). A *trust system* is a triple $(\mathcal{D}, \mathrm{T}, \mathrm{s})$:

1. **Dependency structure** $\mathcal{D}$: a multicategory. Objects are *claims*. A multimorphism $f : y_1, \ldots, y_n \to x$ is a *checking step*: a derivation of x from premises $y_1, \ldots, y_n$ that someone (or something) performed. The 0-ary case $f : () \to x$ is a *ground check* — a check with no premises (reading a source chunk, a Lean compilation, a reproduced computation).
2. **Check-poset T**: a bounded *chain* (linear order with ⊥), ordered by *strength of check*, so meets and joins are min and max. All three deployments are finite chains. [Corrected from "bounded lattice" by the Lean formalization: Lemma 2.2 (i) needs sups *attained*, which finiteness gives in a chain but not in a general lattice — the join of two incomparable elements is attained by neither. Lemma 2.2 (ii) holds in any bounded lattice.]
3. **Grading** $\mathrm{s} : \mathrm{Mor}(\mathcal{D}) \to \mathrm{T}$, assigning to each checking step the strength of the check it actually received.

T is the *deployment parameter*. It is indexed by how knowledge arrives:

deployment	arrival mode	characteristic failure	T
Clio	browse-agent summaries	phantom *content* (LGV determinant that isn't in the paper)	agent-summary < abstract < deep-read < verified-quote
MacBeth	same + feeds	*identity* (wrong paper for a name; withdrawn paper)	+ peer-reviewed, + refutation grades on dead ends

deployment	arrival mode	characteristic failure	T
loop experiment	RAG verbatim chunks	*windowing* (true theorem, hypotheses in an unretrieved chunk)	recalled < rag-summary < chunk-read < context-read < paper-read

`recalled` (claims from training memory, no source behind them) exists only in the third system and sits at the bottom — the check performed was *no check at all*.

Definition 1.2 (derivation). A *derivation* δ of a claim x is a finite tree: a multimorphism $f : y_1, \ldots, y_n \to x$ together with derivations δ_i of each y_i. (The base case is a ground check, $n = 0$.) Formally, derivations of x are the multimorphisms into x in the free multicategory on the underlying graded graph — but "finite labelled rose tree whose root produces x" is exact.

Definition 1.3 (grade of a derivation, canonical trust). The grade of $\delta = (f; \delta_1, \ldots, \delta_n)$ is the weakest link:

$g(\delta) = s(f) \wedge \bigwedge_i g(\delta_i)$ — the meet of ALL step-grades in the tree.

The *canonical trust* of a claim is the best derivation:

$\tau(x) = \bigvee \{ g(\delta) : \delta \text{ a derivation of } x \}$, with $\bigvee \varnothing = \bot$.

So τ is the widest-bottleneck value of x in the semiring $\langle T, \vee, \wedge \rangle$. This is a known provenance-semiring instance (Green–Karvounarakis–Tannen, PODS 2007; the fuzzy/max-min specialization) — the semiring is not the contribution.

Standing hypotheses. $\mathcal{D}$ is *finite* and *well-founded*: the relation "y is a premise of some checking step into x" admits no infinite descending chain — equivalently, every claim's justification bottoms out in ground checks. Both hold in all deployments (registries are finite JSON trees; citations bottom out in sources on disk). Finiteness makes all sups attained; well-foundedness is *load-bearing*, see Remark 2.3.

---

2. Lemma 1: the certificate lemma

Definition 2.1 (sound assignment). An assignment $t : \mathrm{Ob}(\mathcal{D}) \to T$ is *sound* if for every claim x with $t(x) > \bot$ there exists a checking step $f : y_1, \ldots, y_n \to x$ with

$t(x) \le s(f) \wedge \bigwedge_i t(y_i)$. (†)

This is exactly the **boundary rule** as deployed: a registry node may claim level ℓ only if the step-check reached ℓ and every non-dead-end child is at least ℓ. (†) is node-local — checkable without seeing the whole tree. The validators (`registry_validate.py`, `citation_check.py`) are certificate checkers: they verify (†) at each node; they never compute τ.

Lemma 2.2 (certificate lemma). Under the standing hypotheses: (i) τ is sound; (ii) every sound t satisfies $t \le \tau$ pointwise. Hence τ is the greatest sound assignment.

Proof. (i) Suppose $\tau(x) > \bot$. Since $\mathcal{D}$ is finite and T is a *chain*, the sup defining $\tau(x)$ is attained: $\tau(x) = g(\delta)$ for some $\delta = (f; \delta_1, \ldots, \delta_n)$. (Attainment is where linearity works; it fails in a general lattice.) Then

$\tau(x) = s(f) \wedge \bigwedge_i g(\delta_i) \le s(f) \wedge \bigwedge_i \tau(y_i)$,

since each $g(\delta_i) \le \tau(y_i)$ by definition of τ. So (†) holds at x with witness f.

(ii) By well-founded induction on the premise relation. Let t be sound and x a claim; if $t(x) = \bot$ there is nothing to prove. Otherwise pick f as in (†). Each premise y_i is below x in the well-founded order, so by induction $t(y_i) \le \tau(y_i)$, and by finiteness choose δ_i with $g(\delta_i) = \tau(y_i)$. Then $\delta = (f; \delta_1, \ldots, \delta_n)$ is a derivation of x with

$$g(\delta) = s(f) \wedge \bigwedge_i g(\delta_i) = s(f) \wedge \bigwedge_i \tau(y_i) \ge s(f) \wedge \bigwedge_i t(y_i) \ge t(x),$$

so $t(x) \le \tau(x)$. ∎

Remark 2.3 (well-foundedness is load-bearing). Drop it and (ii) fails: take two claims x, y with checking steps $y \to x$ and $x \to y$ both graded $\top$ and no ground checks. Then $t \equiv \top$ is sound but $\tau \equiv \bot$. *Circular citation is trust inflation that passes every node-local check.* The deployed validators are immune only because JSON registries are trees; a generic validator must check acyclicity (or well-foundedness of the premise relation) explicitly. This hypothesis was invisible in all three deployments precisely because the artifact format enforced it by accident.

Remark 2.4 (fixpoint reading). Let $\Phi(t)(x) = \bigvee_f [\, s(f) \wedge \bigwedge_i t(y_i) \,]$ (join over checking steps into x, ground checks contributing s(f)). Then t is sound iff $t \le \Phi(t)$, and τ is simultaneously the greatest post-fixpoint and — by well-foundedness — the least fixpoint reached by Kleene iteration from $\bot$. Consequences, all already observed operationally:

- **Trust inflation** = claiming $t \not\le \tau$: some node fails (†) along every derivation. The phantom LGV determinant was exactly this — a load-bearing claim whose every derivation passed through a leaf whose actual grade was $\bot$ (the check never happened).
- **`unclassified` needs no mechanism.** It is a leaf graded near $\bot$; the $\wedge$ in g caps every derivation through it. "The boundary rule caps everything above an unclassified node" is not a rule — it is the semiring.
- **`dead-end` and `in-progress`** are claims deleted from $\mathrm{Ob}(\mathcal{D})$ for the purpose of (†) (dead ends carry their own grading, see §5).

---

3. Lemma 2: morphisms and opaque restriction

Definition 3.1 (morphism of trust systems). A morphism $(F, \varphi) : (\mathcal{D}, T, s) \to (\mathcal{D}', T', s')$ is a multifunctor $F : \mathcal{D} \to \mathcal{D}'$ together with a monotone map $\varphi : T \to T'$ that is **strict at bottom** ($\varphi(\bot) = \bot'$) and *laxly compatible* with grading:

$$\varphi(s(f)) \le s'(F\ f) \quad \text{for every checking step } f.$$

(Lax, not strict: the target system may re-check an imported step and grade it higher; it may never grade it higher *for free* — that is what the inequality forbids at the level of s, and Lemma 3.2 propagates to τ. Strictness at $\bot$ — "a morphism maps *unchecked* to *unchecked*" — was found missing by the Lean formalization: without it, transport fails on a claim with no derivation, whose $\varphi(\tau) = \varphi(\bot)$ could exceed the target's bottom.)

Lemma 3.2 (transport of trust). For any morphism (F, φ) and claim x:

$$\varphi(\tau(x)) \le \tau'(F\ x).$$

Proof. Induction on derivations. For $\delta = (f; \delta_1, \ldots, \delta_n)$ a derivation of x, $F\,\delta = (F\,f; F\,\delta_1, \ldots, F\,\delta_n)$ is a derivation of F x, and

```
g′(F δ) = s′(F f) ∧ ⋀ᵢ g′(F δᵢ)
        ≥ φ(s(f)) ∧ ⋀ᵢ φ(g(δᵢ))     (lax compatibility, induction)
        ≥ φ( s(f) ∧ ⋀ᵢ g(δᵢ) )       (monotone maps are lax for ∧)
        = φ(g(δ)).
```

Taking sups over δ: $\tau'(F\,x) \geq \bigvee_\delta g'(F\,\delta) \geq \bigvee_\delta \varphi(g(\delta))$. If Der(x) is nonempty, attainment (T a finite chain) gives $\bigvee_\delta \varphi(g(\delta)) = \varphi(\bigvee_\delta g(\delta)) = \varphi(\tau(x))$; if Der(x) is empty, $\tau(x) = \bot$ and strictness at bottom gives $\varphi(\tau(x)) = \bot' \leq \tau'(F\,x)$. ∎ (Both case splits were found by the Lean formalization; the original proof glossed them.)

Note where laxness earns its keep: monotone φ satisfies only $\varphi(a \wedge b) \leq \varphi(a) \wedge \varphi(b)$, and that is the direction the proof needs. Nothing requires φ to preserve meets — interface maps between check-posets in the wild (extraction ladder → registry ladder) do not.

Definition 3.3 (opaque restriction). Let ($\mathcal{D}$, T, s) be a trust system and $R \subseteq \mathrm{Ob}(\mathcal{D})$ a set of *visible* claims (in practice: the stated theorems of an external paper; the roots of another agent's registry). The *opaque restriction* $\mathcal{D}|R$ has objects R and no checking steps except ground checks $() \to x$ for $x \in R$, graded by the *interface*: a monotone $\varphi : T \to T'$ into the consuming system's poset, with the ground check into x graded $\varphi(\tau(x))$ — or, when the consumer cannot compute τ (it cannot see inside the paper), graded $\varphi(e)$ where e is the *extraction level*: the strength of the consumer's own check on the visible claim.

Corollary 3.4 (the two conventions are one construction). A citation is a ground check in the consuming registry: the image under an opaque restriction of an external trust system, graded in the extraction poset and transported along the interface map φ : T_extraction → T_registry. The deployed rule "a source below `context-read` blocks `proved`" is exactly the statement that φ sends everything below context-read strictly below `proved` — and then Lemma 2.2 (†) does the blocking with no further mechanism. The citation convention and the registry convention are not analogous systems; they are the same system on either side of one morphism.

Corollary 3.5 (cross-agent citation). Clio citing a MacBeth registry node is composition of morphisms: MacBeth's system → opaque restriction at his registry roots → Clio's system along φ : T_MacBeth → T_Clio. Lemma 3.2 composed twice gives the safe import grade. This is Phase 4 of the original trust-boundary design, obtained rather than designed.

Mai nafka minah. A fourth deployment is now an instantiation: choose T, point at the dependency structure, and one generic validator — parametrized over (T, φ), checking (†) node-locally plus well-foundedness (Remark 2.3) — replaces the per-agent Python ports. Writing it is simultaneously the reference implementation and the strongest evidence the abstraction is right.

Built (2026-07-04, `code/trustcheck/`). The generic validator exists: one script plus three JSON descriptors (loop, Clio, MacBeth), each descriptor exactly the (T, φ) data — chain, extraction chain, blocking thresholds of φ, evidence fields, certificate format. Regression-identical verdicts with all three deployed ports on their own registries and source indexes. Two things the ports could not say: (i) the explicit cycle check found that the one-hop rule for cross-registry `shared` stubs does NOT exclude citation cycles — a two-registry cycle passes every node-local check in the deployed ports and is rejected by the generic validator with the cycle printed (Remark 2.3 made operational; Lean `cyc` realizable in production data); (ii) the `certificate-gap` report mechanizes the §4 hand-trace: on the G13 layer-4 files it classifies every outgoing reference as cached-ρ / broken-cache / mode 2 (resolves extensionally, fails the certificate format — 40+ backfill candidates, including Warnaar's source.tex 2672–2687 with the line range length-checked and Uncu's main.tex at its exact corpus path) / mode 1 (unresolvable). The $t < \tau$ gap is now a number a machine prints, not a hand-trace.

4. Theorem 3: consolidation preservation (proved, Lean-verified)

The memory pipeline scratch → synthesis → proof is a chain of container morphisms: shapes forward (claims restated), positions backward ("where did this come from?"). A citation is the cached value of the position pullback.

Definition 4.1 (consolidation). A *consolidation* $C : (\mathcal{D}, T, s) \to (\mathcal{D}', T, s')$ over the same check-chain consists of:

- **shapes forward**: a map on claims $x \mapsto Cx$ and on checking steps $f \mapsto Cf$ with premises corresponding ($\text{prems}(Cf) = C(\text{prems } f)$) and grades not dropping ($s(f) \leq s'(Cf)$) — i.e. the forward data of a morphism (Definition 3.1) at $\varphi = \text{id}$;
- **positions backward**: for every claim x and every checking step $f' : y'_1,\ldots,y'_m \to Cx$ of the *target*, a chosen step $\rho(f') : y_1,\ldots,y_m \to x$ of the *source* with premises corresponding ($y'_i = Cy_i$) and the target grade *cached*: $s'(f') \leq s(\rho(f'))$.

ρ is the position map; a citation is precisely its cached value at one step. Note ρ is indexed by the preimage x: when consolidation merges claims, every justification of the merged claim must pull back on *each* side — merging claims of unequal trust is not a consolidation.

Theorem 4.2 (consolidation preservation). If C is a consolidation and both premise relations are well-founded, then $\tau'(Cx) = \tau(x)$ for every source claim x. (Lean: `tau_consolidate`.)

Proof shape. Forward inequality $\tau(x) \leq \tau'(Cx)$: Lemma 3.2 (trust transport) at $\varphi = \text{id}$. Backward inequality $\tau'(Cx) \leq \tau(x)$: well-founded induction on the *source* only — attainment (chain!) picks the target's best step into Cx, ρ pulls it back, the premise correspondence re-indexes the inductive hypothesis, and the cached-grade condition closes the bottleneck bound. ∎

Non-trivialization. Definition 4.1 never mentions τ: it is a bisimulation-style, node-local condition (the validators can check it step by step), and preservation is a *theorem* about it — not "genuine position map" defined as "preserves τ" (too weak, circular), nor "ρ a section of C on the nose" (too strong: real synthesis merges and rephrases; ρ only needs to exist per preimage, C may be many-to-one).

Theorem 4.3 (consolidations compose; Lean `Consolidation.comp`, `tau_consolidate_comp`). Consolidations have identities and compose: shapes forward through both factors, positions backward through both — ρ's compose *contravariantly* (a citation of a citation resolves outermost-first) — and the caches chain by transitivity of $\leq$. Hence a pipeline scratch → synthesis → ... → proof preserves τ end-to-end iff a consolidation exists per layer, and a multi-layer degradation is the failure of (at least) one factor to be a consolidation at all. (Category laws hold but are not stated in Lean: equality of consolidations would compare membership-indexed ρ fields — the `Deriv`-extensionality issue — and every *use* goes through τ values, where the laws are invisible.)

The counterexample direction, corrected by a hand-trace (2026-07-04). The earlier reading of `synthesis-layer4.md` ("shape-only, τ degrades to `recalled` everywhere") conflated two distinct failure modes, which Lemma 2.2 separates:

1. **ρ absent** — the true shape-only case. Formal shadow (Lean `ground/layer4`): one claim with a `hi`-graded ground check consolidates shape-only into a system with no steps; τ falls from `hi` to $\bot$ (`ground_tau_hi`, `layer4_tau_bot`). No consolidation between them exists, and none may: Theorem 4.2 would force $\tau =$ `hi` downstream.

2. **ρ present but cached in prose** — the actual state of the loop experiment's `synthesis-layer4.md`, established by hand-tracing G13 (the d=10 (4,3,3) theorem) through every layer: all cited seed/verifier files exist, Warnaar's Prop_finiteform sits at exactly the cited lines (source.tex 2672–2687), Uncu's main.tex resolves at the verifier's N2 path. A consolidation *exists* at every layer, so by Theorem 4.2 τ is preserved. What the citation checker measured ("zero machine-resolvable citations") is the cache *format*: not one link on the path is written as `slug/chunk_NNN`, from the seed scratch files onward. The deployed validator computes a *sound* assignment (Lemma 2.2 (ii)), hence a lower bound $t \leq \tau$ — and t collapses to $\bot$ on every literature-facing premise it cannot parse. The measured degradation is the gap between certified trust t and canonical trust τ, not a drop in τ.

The two modes converge after one more layer: a consolidator that reads only machine certificates treats unparseable ρ as absent ρ, and mode 2 becomes mode 1. Prose provenance is one compression step from gone.

Operational corollary (sharpened). SYNTHESIZE-SKILL.md's backfill-by-use duty is exactly "carry ρ *in the validator's format*": it is not hygiene, and it is not (usually) restoring lost τ — it is closing the certificate gap $t < \tau$ before the next consolidation makes the loss canonical.

5. Remark: negative provenance

Dead ends with refutation grades are the same construction applied inside the order — positive claims are justified from *above* a threshold, negative claims *within* a refutation sub-poset (MacBeth's deployment already grades them). Wrong reasons silently prune live branches, so refutation grades matter operationally: the frontier includes judgment-level dead ends, and a *recorded* wrong claim cannot be re-hallucinated by the next consolidation, where a deleted one can (field report, "corrections" refinement). Whether this is bi-Heyting structure or just a remark is open.

6. Preserved disagreement (do not force consensus)

Whether the directed-container framing yields theorems or only language over "labelled rose trees with path lookup". Evidence for: `ProofSearchN.lean`'s comonad laws hold with content — duplicate = full sub-search at every node = exactly what a resumed session reads; and Definition 1.2's derivations *are* the positions of that container. Burden of proof: a theorem that USES duplicate non-trivially. Theorem 3 was the natural candidate (consolidation as a container morphism is where positions do work).

Outcome (2026-07-04, honest). Theorem 4.2 is proved, and its Lean proof does NOT use duplicate: plain well-founded induction on the source premise relation, with a per-step position map. "Shapes forward, positions backward" earned its keep as the *definition* — ρ is genuinely a container-morphism position map, and finding Definition 4.1 was guided by that reading — but the *proof* is trees and path lookup.

Second candidate decided (2026-07-04, later the same day). The composition story does not change the verdict. `Consolidation.comp` is proved (Theorem 4.3) and uses nothing comonadic: ρ's compose as plain functions between step types, caches by transitivity of $\leq$, premises by `map_map`. This is *cofunctor* composition in the DCont-as-category dictionary (`scratch/M4-plan.md`: DCont morphisms are cofunctors, not functors), and cofunctor composition never consumes duplicate. Two natural candidates have now both declined the comonad.

Where duplicate DOES carry a theorem in this ecosystem is the validator side, not the consolidation side: `TrustBoundary.lean`'s `ok_sub` (duplicate-stability of the boundary rule) is a genuine one-line transport along D4 — *within-registry* decomposition. The honest split: duplicate governs provenance *within* one

system (subtree at a position); consolidation between systems composes cofunctorially, no comonad consulted. They would meet only if the whole pipeline were internalized as a single container whose positions are cross-layer citation paths — a reframing no theorem has yet demanded. Verdict for the paper: for τ, the containers supply language, not lemmas; the duplicate-shaped content lives in `ok_sub`. Say both.

7. Prior art (search completed 2026-07-04)

Semirings — cite, not claim. Green–Karvounarakis–Tannen, *Provenance semirings*, PODS 2007, is the canonical reference. For the min-max semiring specifically: Brinke–Dawar–Grädel–Pago, *Preservation Theorems in Semiring Semantics* (arXiv:2605.10829, **deep-read**) — Thm 4.1 proves the classical preservation theorems hold for all lattice semirings, hence for min-max; Lemma 3.3 shows they fail for tropical/Viterbi. Cite it as "min-max is a well-studied semiring for which classical preservation theorems hold" — NOT as "min-max is a provenance semiring": in their framework the provenance semirings are the polynomial ones ($\mathbb{N}[X]$, absorptive polynomials), and min-max is an application semiring. [Corrected from the agent-summary version of this paragraph, which conflated the two.] Their definition (§2.1, verified quote): "A totally ordered set $(S,\leq)$ with least element s and greatest element t induces the min-max semiring (S, max, min, s, t)" — *totally ordered*, exactly the chain requirement the Lean formalization forced independently (Lemma 2.2 (i)); the two constraints corroborate each other. Living extension of GKT: Grädel–Tannen (arXiv:2412.07986, FO logic with negation via dual-indeterminate polynomials). Widest-bottleneck over paths: Grahne–Thomo (arXiv:2001.09864), graphs rather than derivation trees.

Closest neighbour — must be addressed in the paper. The Assurance 2.0 line, Bloomfield–Rushby (arXiv:2205.04522; confidence propagation through assurance-case trees in arXiv:2409.10665 and arXiv:2604.00034, the latter **deep-read**). They want exactly a sound propagation from leaf evidence to root claim, but their vehicle is probability logic with Fréchet bounds (§3.2; e.g. "doubt(parent) = $\prod_i$ doubt(subclaim$_i$)", Eq. 5) — and the propagation *branches by argument structure* into four case-type schemas (diversity, partition, containment, cumulative), with explicit independence assumptions (§3.2.3) whose variation swings the same inputs from 0.55 to 0.997 (their Table 1). Precision matters on one point: `min` DOES appear (the Fréchet upper bound for intersection, §3.2.5) — but on real-valued confidences. What they do not have is min propagation over a *discrete grade poset*; our boundary rule is a node-local soundness condition on a chain: simpler, deployment-agnostic, no independence assumptions, one rule rather than four schemas. Their "dialectical" perspective handles defeaters as a separate four-category qualitative overlay (§3.3) applied only after logical soundness — not integrated into the propagation, which is where §5 differs. Quantitative argumentation (arXiv:2603.06067 bipolar gradual semantics; arXiv:2207.02258 strength-based admissibility) has strength propagation with parametrized aggregation, never fixed at min over a check-poset.

Consolidation loss — motivation exists, formalization doesn't. A cluster of 2025–2026 agent-memory papers (arXiv:2603.19595 All-Mem, arXiv:2604.26622 OCR-Memory, and others) explicitly names "irreversible information loss from summarization-based compression" as the failure mode — uniformly informal vocabulary, no morphism, no preservation question. Practical trust-DAG and provenance-through-delegation systems exist (arXiv:2604.27132 TRUST; arXiv:2603.00960 LDP, which notes "noisy provenance degrades synthesis quality"; arXiv:2604.22755 citation-backed RAG) — engineering realisations, no formal model. The Spivak functorial-data-migration line (arXiv:1009.1166 etc.) never touches provenance or trust grades.

Honestly scoped contributions, confirmed open after search: (a) the check-poset as deployment parameter — no existing work parametrizes the *grade structure* by how knowledge arrives; (b) consolida-

tion as a lossy endofunctor with a categorical preservation condition — the engineering literature agrees the loss happens, nobody formalizes what "loses" means; (c) graded negative provenance integrated into the same order as positive trust — assurance cases keep defeaters as an overlay, argumentation grades them but in a different structure.